



Dynamic Deep Stock Price Prediction Network with Support of Multi-Source Information

Gayathri Ravi^{1*} Devi Suganya Selvakumar¹

¹*Department of Computer Science, Vellalar College for Women, Erode – 638012, Tamilnadu, India*

* Corresponding author's Email: 2025gayathriphd@gmail.com

Abstract: Stock market trading relies on quick, accurate information, but predicting stocks is difficult due to various unpredictable factors. Several Deep Learning (DL) models have been developed to enhance stock prediction efficiency. While some DL models perform well with single-source data, they often show lower accuracy when handling multi-source stock market information. Multilevel Graph Attention Model (ML-GAT) integrates multi-source data by leveraging Long Short-Term Memory (LSTM) networks and Bidirectional Encoder Representations from Transformers (BERT) for feature learning and embedding within stock. This model integrates financial data, news articles and Wikidata to support stock market prediction. Despite lacks to consider multiple source of indicators like technical, sentiment and fundamental indicators to identify stock trends, ML-GAT finds difficulty in leveraging heterogeneous data sources, impacting the models performance. In this paper, Dynamic Deep Stock Price Prediction Network (DDSPPnet) is developed using Enriched Multi-Level Graph Attention model (EMLGAM) to solve above-mentioned issues for stock prediction. In this approach, stock market data features are derived from four key sources like price data, quantitative data (financial and technical index factors), sentiment data (news) and relational data (Wikidata). Initially, price data features are extracted using an LSTM module to capture long-term dependencies in short-term price fluctuations. Next, quantitative data features are extracted using Gated Recurrent Units (GRU) to efficiently capture temporal dependencies and trends over time. For sentiment data, features related to public opinion (news) are extracted using BERT to capture stock market movements and predict market behaviour. The relational data leverages public Wikidata to extract enterprise relationship information using an improved relation extraction model. This improved relation extraction model utilizes Multi-Variate Hawkes Process (MVHP) to extract initial stock correlations that are then combined with Wikidata's relationship data for improved stock prediction. The extracted features are embedded together and inputted to EMLGAM for stock price prediction. The test results demonstrate that the proposed model achieves an average accuracy of 0.893, 0.904, 0.913, 0.903, along with an average F1-Score of 0.896, 0.901, 0.906, 0.900 on India's National Stock Exchange (NSE) and Bombay Stock Exchange (BSE), Standard & Poor's 500 Index (S&P 500) and National Association of Securities Dealers Automated Quotations (NASDAQ-100) datasets respectively, outperforming LSTM, GRU, BERT, LSTM-GRU, BERT-LSTM and BERT-GRU models.

Keywords: Stock market, Long short term memory, Multi-variate hawkes process, Gated recurrent units, Graph convolution network.

1. Introduction

Advancements in information technology have revolutionized financial markets, integral to national economies [1]. Stock trading, now essential for businesses, enables capital raising and share trading [2]. Investors profit by buying low and selling high, making the stock market vital for global economic

growth [3]. It fosters company expansion, wealth-building, and drives interest in stock value prediction [4]. Stock market prediction involves forecasting future price movements of stocks or overall market, influenced by economic indicators, company performance, market sentiment, geopolitical events, and social trends [5, 6]. While accurate predictions can yield significant returns, the dynamic and unpredictable nature of markets makes this task

challenging. Traditional statistical models like Average (ARIMA) [7] and Generalized Autoregressive Conditional Heteroskedasticity (GARCH) [8] have been widely used for financial time-series forecasting, effectively capturing trends, seasonality, and volatility. However, these models struggle with market shifts, non-linearity, and extreme events, causing inaccurate predictions during high volatility.

In recent years, DL has become crucial for stock market prediction [9]. Techniques like Convolutional Neural Networks (CNNs), Graph-Based CNNs, Recurrent Neural Networks (RNNs), and Long Short-Term Memory (LSTM) networks being widely used. RNNs are effective for mining time-series data in stock prediction [11]. But, they face challenges like gradient dissipation and explosion. LSTMs address these issues by managing long-term memory through cell gate mechanisms improving prediction accuracy [11]. Despite DL models' success in financial forecasting, integrating news, events, and company data is crucial for improved accuracy [12, 13]. Many existing models focus on partial data, neglecting the broader context of stock market dynamics.

To improve stock market predictions, integrating multisource data from diverse sources is essential. This includes financial metrics, news, social media content, supply chain insights, geopolitical events, and market microstructure data [14]. Although advanced graphical models have been developed, many approaches simply aggregate data into graphs without fully leveraging the complex context of stock market behavior [15]. As a result, effectively mining and integrating multisource data remains a key challenge in stock market analysis.

To address this issue, the Multilevel Graph Attention Model (ML-GAT) [16] was developed for stock prediction by combining LSTM and BERT for feature learning and embedding into stock graphs. Using attention layers, it filters and aggregates diverse data to improve prediction accuracy. The model integrates financial market data, news articles, and from Wikidata to enhance stock market prediction. While there is multiple source of indicators like technical indicators (moving averages, Relative Strength Index), sentiment indicators (financial news, earning reports social media trends) and fundamental indicators (knowledge graph from Wikidata, macroeconomic indicators) to identify trends, assess risks and inform investment decisions, the ML-GAT model has limitations in fully leveraging these heterogeneous data sources. Its current architecture may struggle to align the temporal, textual, and relational aspects of multi-

source data potentially leading to suboptimal performance. Hence, in this paper, DDSPPnet is proposed to handle the large number of features, determines the effective correlative between the stocks and selects key indicator from multiple sources to enhance stock trend forecasting accuracy. In this model, Key stock data factors include price, quantitative, sentiment, and relational data. The price data are extracted using LSTM module to capture both and long-term dependencies in price variations. Quantitative data including financial and technical indices are processed using GRU model to capture temporal dependencies and trends. Sentiment data reflecting public opinion from news are analyzed using BERT encoder model to extract features. Relational data leverages public wikidata in which the features are retrieved using improved relation extraction model. Improved relation extraction module devises MVHP to extracts initial stock correlations. The derived correlation will be fused with the Wikidata's relationship data to enhance prediction accuracy. The extracted features are integrated together and fed into EMLGAM which uses GCN to extract and analyze relational data. The concatenated feature vector is fed to fully connected (FC) layer that uses a feature map matrix and weights of inputs to predict stock.

The residual sections are planned as follows: Section II covers the related works. Section III presents the methodology of DDSPPnet model. Section IV portrays its efficiency. Section V concludes the study and suggests future enhancements.

2. Literature survey

In [17], a Multi-scale CNN (MS-CNN) model was presented to efficiently extract the stock trend features and predict the stock. But, it selects over-estimated value in some cases leading to longer training time. Multi-Source data with Sparrow Search Algorithm and LSTM (MS-SSA-LSTM) was constructed [18] for the prediction of stock prices. But, lower accuracy and f1-score results when training on larger datasets.

Multimodal Early Fusion Transformer (MEFT) model was developed [19] for accurate stock price prediction using an early fusion strategy to effectively integrate information across modalities. But, the efficiency of early fusion approach was not consistent across all stocks especially for sector-specific features. Additionally, it fails to consider fundamental and sentiment indicators lowering the models performance. In [20], a graph convolutional

semi-supervised model (PA-GCN) using graph attention mechanism and Elu activation function for stock prediction. But, validation on a small dataset limits the models performance.

In [21], a Hybrid Information Mixing Module (HIMM) was created using GRU and BERT for stock movement prediction. But, the model was relied only on two modalities like price and textual data, other influential factors were also excluded which lowers the models ability to capture the complex market dynamics. Graph CNN (GCNN) was developed [22] for classifying the financial stocks and analyzing market development. Nevertheless, lower accuracy was resulted when trained on larger dataset.

Hybrid machine learning model was devised [23] for stock market prediction (HM-SMF) using chaos-enhanced firefly bowerbird optimization (CEFBO) algorithm for optimal feature selection and hybrid multi-objective capuchin with RNN (HC-RNN) for prediction. But, higher RMSE was resulted as this model fails to handle non-sequential data. In [24], stacking ensemble model was developed for stock prediction by integrating news headlines multi-variate time series data. However, this model results in high time and space complexity.

A stock market prediction approach using DL (SMP-DL) was devised [25]. LSTM and Bidirectional Gated Recurrent Unit (BiGRU) were used for stock trend forecasting. But, training time was high when trained on newly generated anonymous data. In [26], a stock prediction model was presented by integrating multi-view stock data features with dynamic market correlation information (MDF-DMC). However, Lower accuracy and F1-score resulted from training challenges on an imbalanced dataset.

3. Proposed methodology

In this section, the complete framework of DDSPPnet is illustrated. Fig. 1 depicts the pipeline of the proposed methodology. Table 1 depicts the notation table.

3.1 Feature extraction module

In stock market graph modelling, each company's stock is a node, with features representing its state and price movements. These features evolve based on market dynamics. The model predicts stock changes using historical prices, financial news, key indices, and Wikidata-based stock correlations. This model process different features extraction module which are listed below.

Table 1. Lists of notations

Notations	Description
CR^t	Rate of change stock price at time t
$p_x^t; p_x^{t-1}$	Closing stock prices of stock x at time t and $t - 1$
p_x^t	Historical Price Data
S	Sequence Length
D	Feature dimension
Z	Sequential Embedding
h	Hidden Layer
p, Z	Text Feature embedding in LSTM
l	Embedding Size in LSTM
m	Quantitative Data
t	Time Period
N	Time Steps
Y	financial and technical features
r_t	Reset Gate
u_t	Update Gate
b, W	Bias, Weight
σ	Sigmoid Function
z	Previous State
m, O	Text Feature Embedding in LSTM
R	Embedding size in GRU
[CLS]	Classification Token
[SEP]	Separation Token
T	Stock Price News
E	Feature Dimension of Each Text
n_i^t	Related news item
r	Reset gate
$\mathcal{K}, F$	Text Feature Embedding in BERT
Q	Embedding size in BERT
G, V, E, T	Graph, Node, Edge, Stock Type
$\varphi(v); \phi(e)$	Node Mapping; Edge Mapping
$T_v; T_e$	Node Type ; Edge Type
$R_1; R_2$	Relations in meta-path
$\mathcal{E}, q_f$	Event Type; Event Sequences
$t_x^f; e_x^f$	Event Type; Stock Time Stamp
μ_e	Intrinsic Intensity of Event Type
l_v	Infectivity of event type
$A^f; \mu^f$	local infectivity, local natural intensity matrix
X, Y, Z	Wikidata Entities
$R, R_1 R_2$	Various Relation Types in Wikidata
r_k	Relationship Type
$\delta(\cdot)$	Activation Function
$W_g; U_g$	Two learnable parameters
g_l	Gate Control
$a; b$	Target and Source Node
u_{ab}^{rk}	Connection Vector
α	Constant Factor
C_x^{rk}	Summary Information Vector
e_{rm}	Relationship Embedding Vector
$\hat{Y}_x$	Transformation Layer
L	Loss Function

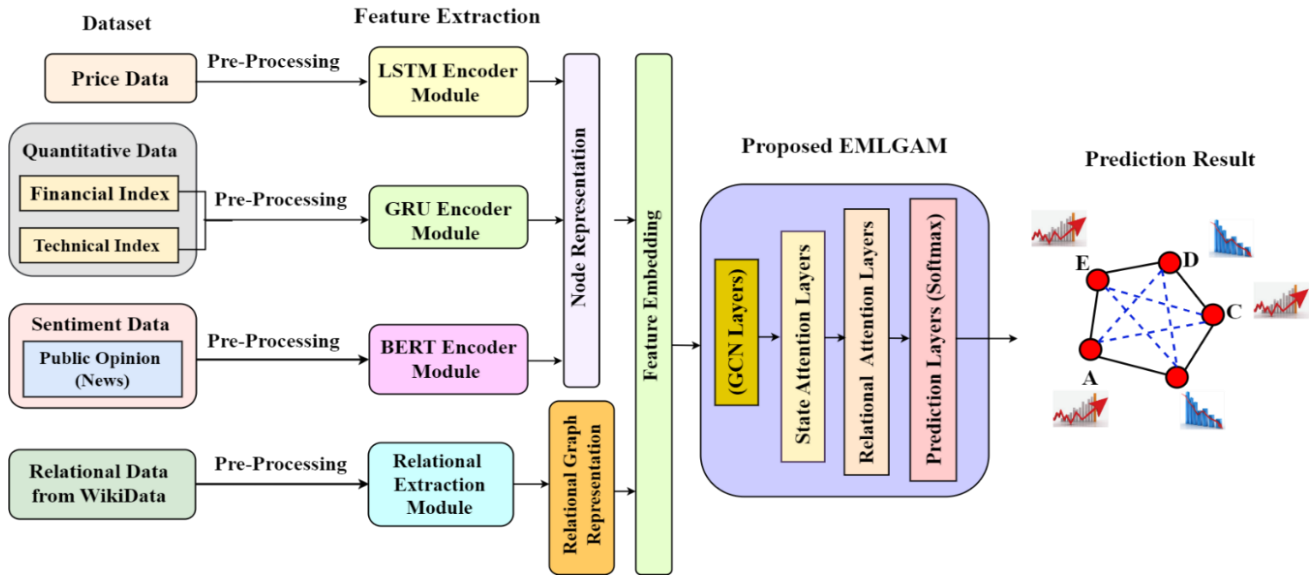


Figure. 1 Diagrammatic depiction of the proposed methodology

3.1.1. LSTM encoder module

In this model, The LSTM module analyzes historical closing prices, capturing long-term market dependencies through price change rates. The rate of change stock price at time t can be computed in Eq. (1)

$$CR_x^t = \frac{p_x^t - p_x^{t-1}}{p_x^{t-1}} \quad (1)$$

Where p_x^t and p_x^{t-1} are the closing prices of stock x at time t and $t - 1$. To pre-process the historical price data, consider historical price change rate of stock x at time step t as $p_x^t = \{p_x^{t-S+1}, \dots, p_x^{t+1}, p_x^t\} \in \mathbb{R}^{S \times D}$ in which S and D represents the sequence length and feature dimension at each time step. The time-series related to price data p_x^t are inputted to the LSTM network and the last hidden state of the output (h_x^t) is used as the sequential embedding (Z_x^t) ($hx^l = Z_x^t$) of the following network.

Fig. 2 depicts the feature extraction from price data. Following Eq. (2)–(5) describes the LSTM structure.

$$i_x^t, f_x^t, o_x^t = f_{\theta^i}, f_{\theta^f}, f_{\theta^o}(W[h_x^{t-1}; p_x^t] + b_{(i,f,o)}) \quad (2)$$

$$c_x^t = i_x^t \odot u_x^t + f_x^t \odot c_x^{t-1} \quad (3)$$

$$u_x^t = \tan(W^{(u)}p_x^t + U^{(u)}h_x^{t-1} + b^{(u)}) \quad (4)$$

$$h_x^t = o_x^t \odot \tan(c_x^t) \quad (5)$$

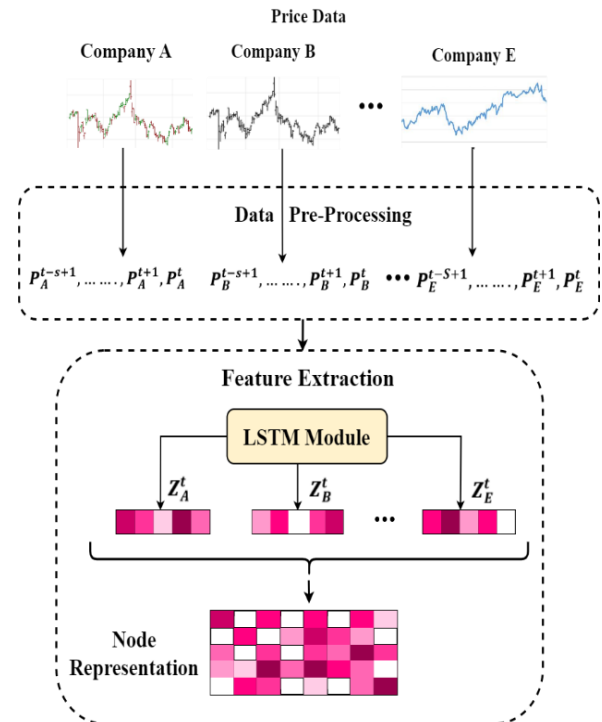


Figure. 2 Feature extraction from price data using LSTM

Simply, it is determined in Eq. (6)

$$Z^t = LSTM(p^t) \quad (6)$$

Here, $p^t = [p_1^t, \dots, p_x^t, \dots, p_K^t] \in \mathbb{R}^{K \times S \times D}$, $Z^t = [z_1^t, \dots, z_x^t, \dots, z_K^t] \in \mathbb{R}^{K \times I}$ represents the sequential embedding of K stocks and I represents the embedding size (the number of hidden units in the LSTM).

3.1.2. GRU encoder module

An improved LSTM version overcomes RNN limitations, simplifying structure and reducing parameters while maintaining performance. In stock prediction, GRU processes time series data from financial indices, technical indicators like moving averages, RSI and Bollinger Bands, capturing critical patterns and trends for forecasting.

Its update and reset gates effectively manage long-term dependencies, making it suitable for stock market prediction. Assume, quantitative data (financials and technical index factor) m is collected from stock x in time period t as $m_x^t = \{M_x^{t-T+1}, \dots, M_x^{t+1}, M_x^t\} \in \mathbb{R}^{N \times Y}$, N and Y represents number of time steps and feature dimension at each time step of financial and technical indices respectively. The GRU is a simplified alternative to LSTM, combining forget and input gates into a single update gate and reset gate. Given the current input m_x^t and previous hidden state h_{t-1} which includes relevant information about the previous node.

Through the information in i_t and h_{t-1} , GRU regulates hidden node outputs via gates, transferring information without a separate memory cell. The update gate u_t (in Eq. (7)) determines how much past information. The reset gate r_t in (Eq. (8)) regulates the past states influence on the current data. The grade of u_t and r_t can be updated and reset at any time t using the Eqs. (7) and (8)

$$u_t = \sigma(W_z [m_x^t, h_{t-1}] + b_z) \quad (7)$$

$$r_t = \sigma(W_r [m_x^t, h_{t-1}] + b_r) \quad (8)$$

Once the gating signal is obtained, the reset gate is determined for reset data which is integrated with the Rectified Linear Unit (ReLU) activation function to obtain the cell memory $\tilde{h}_t$ at time as in Eq. (9)

$$\tilde{h}_t = \tanh(W_h [r_t \times h_{t-1}, m_x^t] b_h) \quad (9)$$

In Eq. (9), r_t denotes the state information of reset gates, $\tanh$ represents the neuron connection weight. If the reset gate is off ($r_t == 0$), then it enables the unit to disremember the former. This allows the unit to understand the initial input data. Fig. 3 illustrates feature extraction from financial and technical indices. In the update phase, the update gate is determined to obtain the final output data h_t at time t which is given in Eq. (10)

$$h_t = (1 - u_t) \times h_{t-1} + u_t \times \tilde{h}_t \quad (10)$$

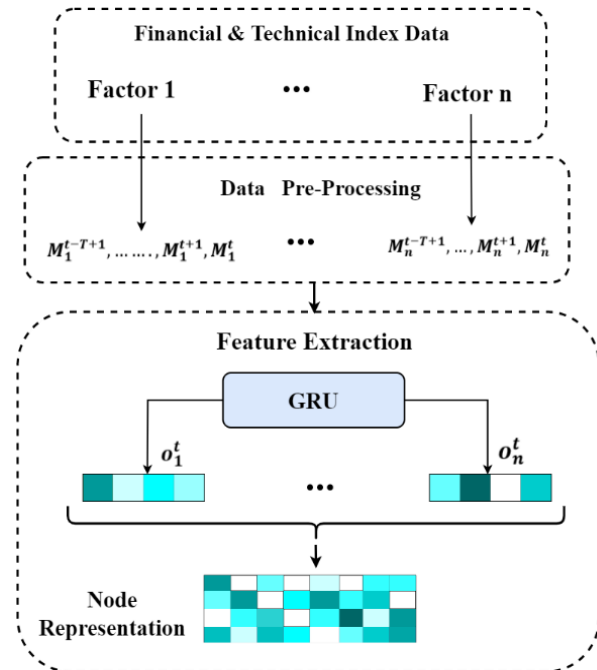


Figure. 3 Feature Extraction from Financial and Technical Indices Data using GRU

The activation h_t of GRU at period t is a linear interpolation between the earlier activation h_{t-1} and the nominee activation $\tilde{h}_t$. In above Eqs. (7)-(10), W_z, W_r, W_h are weight matrices, whereas b_z, b_r, b_h are bias vector. Where, σ represents the sigmoid function ranges controlled between 0 to 1. The update gate u controls the importance of the previous state, with reset gate r for short-term for long-term dependencies. Simply, it can be determined in Eq. (11)

$$O^t = GRU(m^t) \quad (11)$$

Where, $m^t = [M_1^t, \dots, M_x^1, \dots, M_K^t] \in \mathbb{R}^{K \times N \times Y}$ and $O^t = [o_1^t, \dots, o_i^t, \dots, o_K^t] \in \mathbb{R}^{K \times R}$ signifies the text feature embedding vector of K stocks and R represents the embedding size (the number of hidden units in GRU to predict stock movements).

3.1.3. BERT encoder module

BERT's input embedding includes token, segment, and position embedding layers. The token embedding converts each word into a fixed vector, with "[CLS]" marking the start. Segment embeddings distinguish sentence types, marked with "[SEP]", while position embeddings are learned from the text data. Fig. 4 depicts the feature extraction from public opinion data. In this model, BERT is used to extract stock news text data features in the financial field.

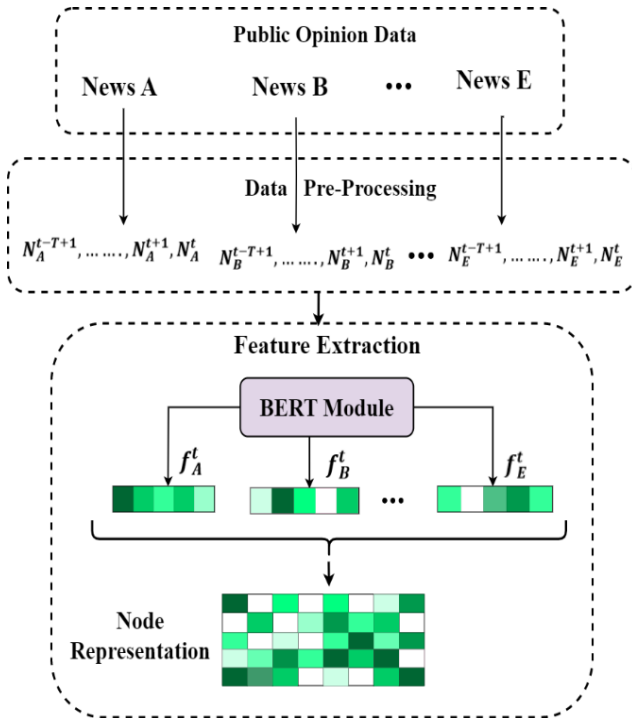


Figure. 4 Feature extraction from public opinion data using BERT

Assume news pieces T is collected from stock x in time period t as $n_x^t = \{N_x^{t-T+1}, \dots, N_x^{t+1}, N_x^t\} \in \mathbb{R}^{T \times E}$ in which T and E represents the number of text sequence and feature dimension of each text respectively.

The BERT-based module pre-processes the input sentence by truncating or completing it, adding the [CLS] token at the start and the [SEP] token at the end for encoding. Related news item n_i^t for stock i is converted to $[CLS] n_i^t + [SEP]$, where the output vector corresponding to [CLS] can represent the semantic feature information of the sentence. The final [CLS] vector in BERT captures the sentence's semantic features after extensive learning. For news text extraction, only the [CLS] vector from the last transformer layer is used, as shown in Eq. (12).

$$F^t = \text{BERT}(\mathcal{K}^t) \quad (12)$$

Here, $\mathcal{K}^t = [N_1^t, \dots, N_i^t, \dots, N_K^t] \in \mathbb{R}^{K \times T \times E}$, $F^t = [f_1^t, \dots, f_i^t, \dots, f_K^t] \in \mathbb{R}^{K \times Q}$ signifies the text feature embedding vector of K stocks and Q represents the embedding size.

3.1.4. Relational extraction module

The relation extraction module extracts enterprise relations from Wikidata, forming a heterogeneous graph of entities and connections. Consider, Wikidata as a heterogeneous network $G = (V, E, T)$, where

each node v and each link e are distributed and their mapping functions $\varphi(v) : V \rightarrow T_V$ and $\phi(e) : E \rightarrow T_E$ are associated. T_V and T_E represent collections of node objects and relation types. The task of relationship extraction module is to learn the d -dimensional latent representation $U \in \mathbb{R}^{|V| \times d}$ ($d \ll |V|$) and capture the structural relationships between the target nodes. Meta-path bootstrapping with up to 2 hops guides random walks in heterogeneous networks using a predefined scheme as $p: V_1 \xrightarrow{R_1} V_2 \xrightarrow{R_2} V_3$. The probability of transition at step x is defined in Eq. (13),

$$\mathcal{P}(v^{x+1} | b_t^x, p) = \begin{cases} \frac{1}{|N_{t+1}(v_t^x)|} & (v^{x+1}, v_t^x) \in E, \Phi(v^{x+1}) = t+1 \\ 0 & (v^{x+1}, v_t^x) \in E, \Phi(v^{x+1}) \neq t+1 \\ 0 & (v^{x+1}, v_t^x) \notin E \end{cases} \quad (13)$$

In Eq. (13), $v_t^x \in V_t$ and $N_{t+1}(v_t^x)$ represent the V_{t+1} type of the field of node v_t^x .

In order to find the initial correlation graph between stocks, MVHP is used. MVHP [27] model the event sequences by capturing both triggering patterns within sequences and clustering structures across them. MVHP improves on standard Hawkes processes by modeling multiple sequences together using structural constraints that uncover clusters. Its structured infectivity tensor captures shared triggers, helping prevent overfitting with sparse event data.

In the given F sequences with $\mathcal{E}$ events types, the f^{th} event sequences is represented as $q_f = \{(t_x^f, e_x^f)\}_{x=1}^{N_f}$, where $= 1, 2, \dots, F$. $t_x^f \in (0, T_f)$ and $e_x^f \in (1, \dots, \mathcal{E})$ are the event type and time stamp of the x^{th} event in q_f correspondingly. T_f and N_f defines the time span and number of events of q_f respectively. Each sequence q_f can be modelled by MVHP. The conditional intensity of events λ_e is given in Eq. (14)

$$\lambda_e(t) = \mu_e + \sum_{v: t_v < t} I_{ee'} \times \varpi(t - t_v) \quad (14)$$

Where, μ_e represents the intrinsic intensity of the event of type e . I_v is the infectivity of type of e' events to type e events. ϖ defines the triggering kernel function quantifying the triggering effects from previous events. $\sum_{v: t_v < t} I_v \times \varpi(t - t_v)$ represents the influence of historical events on the instantaneous rate of event at time t . By utilizing the Eq. (14), the conditional intensity for q_f on e type event at time t is given in Eq. (15)

$$\lambda_e^f(t) = \mu_e^f + \sum_{v: e_v^f < t} I_{ee_v}^f \times \varpi(t - t_v^f) \quad (15)$$

Varied from the conventional multi-dimensional Hawkes task in (14), the model parameters in (15) fall in two categories like (i) a global triggering kernel $\varpi(t)$ reflects the diminishing influence of historical events and is shared across all sequences; (ii) a local infectivity matrix $A^f = a_{ee'}^f$, and a local natural intensity vector $\mu^f = [\mu_1^f, \dots, \mu_\varepsilon^f]$ which are specific to sequence q_f . A^f and μ^f represents into an intrinsic intensity matrix $\mu = [\mu_e^f] \in \mathbb{R}_+^{\varepsilon \times F}$ respectively. In this graph, nodes represent stocks and edges indicate the strength and direction of their stock correlations. Fig. 5 depicts the feature extraction from Wikidata. MVHP captures temporal dependencies via self and mutual-excitation, showing how past events influence future asset movements.

A relation extraction layer refines correlations by quantifying financial relationships, enhancing performance. Refined correlation data from MHVP supports trading strategies, risk management, and predictive analytics, enhancing market insight.

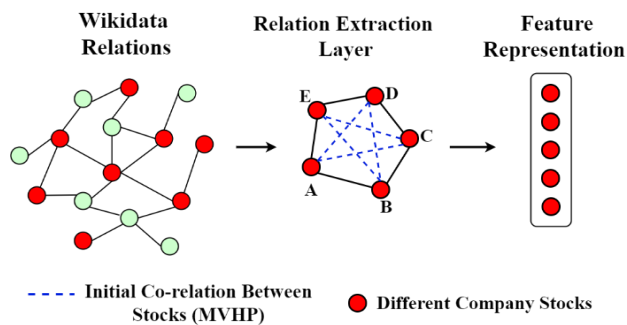


Figure. 5 Feature extraction from Wikidata using improved relation extraction layer

Table 2. Enumeration of first-order wiki relation

No	Relation Index	Description
1	Instance of (P31)	Specifies an entity is a particular type or class, such as stock exchange
2	Location (P276)	Indicates the location where the NSE\BSE is situated
3	Entity Subdivision (P527)	Refers to components or subdivisions within the NSE\BSE
4	Country (P17)	Denotes the country in which the NSE/BSE operates
5	Official website (P856)	Provides the URL of the NSE's official website.

Table 3. Enumeration of second-order wiki relation

No	Index	Description
1	Listed on (P424)	Exchanges where the NSE\ BSE-listed stocks are traded
2	Parent organization (P127)	The organization overseeing or associated with the NSE\BSE
3	Operator (P137)	Entity responsible for managing or operating the NSE\BSE
4	Headquarters location (P159)	The physical location of the headquarters for NSE\ BSE
6	Member of (P463)	Organizations or networks that NSE\ BSE is part of
7	Established by (P112)	Entities or individuals who established NSE\ BSE
8	Regulated by (P611)	Regulatory bodies overseeing NSE\BSE
9	Number of employees (P1128)	Employee count for NSE\BSE
10	Industry (P452)	Specific industry or sector in which NSE\BSE operates.

By utilizing relation extraction module, few types of first-order ($X \xrightarrow{R} Y$) and second-order ($X \xrightarrow{R_1} Y \xleftarrow{R_2} Z$) relation between enterprises corresponding to the selected stocks. X, Y and Z represent entities (Q) in Wikidata while R, R₁ and R₂ denotes different relations properties (P) defined in Wikidata. The details of the first-order and second order relations are provided in Tables 2 and 3 respectively.

3.2 Feature embedding

In this network, the company nodes i.e., final node representation of LSTM, GRU, BERT is upgraded in the isomorphic graph transformed by the relational extraction module.

The main function of the GCN is to exchange information between adjacent nodes, then aggregate the information of adjacent nodes and to finally add it to each node's representation. Node features are key to the success of the graph prediction task, it is effective to aggregate different types of relationship information collected from different nodes.

Through the stock data feature extraction module, the price embedding vector $z_x^t \in \mathbb{R}^h$, financial and technical index embedding vector $o_x^t \in \mathbb{R}^h$ and news text embedding vector $f_x^t \in \mathbb{R}^h$ of stock x at time t can be obtained. Furthermore, since the model is implemented based on a graph neural network (GNN), it is crucial to know the set of neighbour nodes of the

target node i in each relation type. The relation type e of node i as $r_k \in c_x^{r_k}$ and the embedding vector of relationship type r_k as $Q_{r_k} \in \mathbb{R}^d$. The goal is to selectively collect information about different relationships from neighbouring nodes with data and relational graph. Feature embedding task i.e., Z_x^t node representation of LSTM (price data), O_x^t node representation of GRU (Financial and technical indices data) and F_x^t node representation of BERT (News data) with relational graph representation module Wikidata) which is illustrated as in Eqs. (16)-(18)

$$Z_x^t = \{z_a^t, z_b^t, z_c^t, z_d^t, z_e^t\} \quad (16)$$

$$O_x^t = \{o_a^t, o_b^t, o_c^t, o_d^t, o_e^t\} \quad (17)$$

$$F_x^t = \{f_a^t, f_b^t, f_c^t, f_d^t, f_e^t\} \quad (18)$$

These diverse node embeddings of price data, quantitative data and sentiment data are integrated into a relational graph module to enhance the relational context and provide a comprehensive understanding by linking and enriching the data with external knowledge for stock trend representation.

3.3 Design of EMLGAM

In this model, EMLGAM predicts stocks using GCN, state attention, relation attention, and prediction layers. It employs weighted attention mechanisms to capture diverse node relationships, enhancing trend prediction by filtering irrelevant inter-stock connections. This approach addresses the complexity and diversity of relationships, improving accuracy in stock trend forecasting.

GCN Layer (First Layer): IDMGA uses GCN, an extension of CNN for graphs, performing convolution filtering and updating node representations by propagating information between nodes. All three layers of constructed nodes are stacked into the node set V . The set of nodes V are represented as d -dimensional vectors $V \in \mathbb{R}^{(N+R+T) \times d}$, GCN layer on graph can be written as non-linear function $f(V, A)$. When considering the stacking of multiple GCN layer and after exploiting the convolutional operation [28], the GCN can be represented in Eq. (19),

$$V^{l+1} = f(V^l, A) = \delta(AV^l W^l) \quad (19)$$

Where $\delta(\cdot)$ denotes an activation function, which is chosen as LeakyReLU in our experiments. As a general rule, the superscript l indicates the layer

number. $W^l \in \mathbb{R}^{d \times d}$ is the learnable parameters in convolutional filter. Different graph convolution layers have different convolutional filters, which are numbered using l . Multiplying the adjacency matrix is equivalent to adding each node's neighbor feature.

In this approach, GCN aggregates neighbor information with uniform weights and updates features via activation, learning relationships. Laplacian smoothing enhances intra-cluster similarity, but excessive GCN layers cause over-smoothing, blurring features and limiting the capture of long-range dependencies. To address over-smoothing, GCNs use a gating mechanism that aggregates neighbor information, then controls its influence on node updates. The gating mechanism can be calculated in Eqs. (20) and (21),

$$g_l = \text{sigmoid}(W_g V_{l+1} + U_g V_l + b_g) \quad (20)$$

$$V_{l+1} = V_{l+1} \odot g_l + V_l \odot (1 - g_l) \quad (21)$$

Where $W_g \in \mathbb{R}^{d \times d}$ and $U_g \in \mathbb{R}^{d \times d}$ are two learnable parameters. The gate g_l controls the new node representation V_{l+1} update of each layer by considering the node representations generated by previous layer V_l and the current graph convolutional layer V_{l+1} . The gating mechanism preserves local node information after convolution, combining it with global data to improve document graph understanding and reduce over-smoothing in deep GCNs.

State attention layer (Second layer): IDMGA extracts relevant information from adjacent nodes, using attention mechanisms to weight based on relationship type r_k . Before calculating the state attention coefficient, it is necessary to concatenate the relation type embedding vector r_k and the node representations of nodes a and b . The connection vector is denoted as $b \in N_x^{r_k}$. Eq. (22) depicts the formula for calculating the attention coefficient of the relationship type r_k between target node a and b

$$\delta_{ab}^{r_k} = \frac{\exp(\text{LeakyReLU}(\bar{v}_r^T (W_s x_{ab}^{r_k} + b_s)))}{\sum_{m \in N_i^{r_k}} \exp(\text{LeakyReLU}(\bar{v}_r^T (W_s l_{xm}^{r_k} + b_s)))} \quad (22)$$

Where $W_s \in \mathbb{R}^{2h+d}$ and $b_s \in \mathbb{R}$ are the learnable parameters used to compute the state attention coefficient. Based on the calculated state attention coefficients, the output feature of the company relationship is calculated as having a weighted average of all nodes using the Eq. (23)

$$C_a^{r_k} = \sigma \left(\sum_{j \in N_x^{r_k}} (\delta_{ab}^{r_k} W_s z_b) \right) \quad (23)$$

Eq. (23) computes each stock relationship's vector representation, summarizing its information.

The vector $Z_x^{r_k}$ includes summary information from the relationship type r_m of stock x , prioritizing critical information for trading decisions, similar to human investment behavior.

Relation attention layer (Third layer): In this layer attention mechanism is used to assign importance weights to many pieces of sign importance weights to many pieces of information. The information vector $C_x^{r_k}$ of the relationship, the current node representation z_x^t , y_x^t , o_x^t and f_x^t of the stock company and the embedding vector of the relationship type e_{rm} are integrated and denote the concatenated vector as $v_x^{r_k} \in \mathbb{R}^{2h+d}$ as the feature input for the next attention layer. Similar to the previous formula, in the second-level attention layer, the attention coefficient is calculated in Eq. (24)

$$\delta_a^{r_k} = \frac{\exp(\text{LeakyReLU}(\tilde{t}_r^T(W_r v_x^{r_k} + b_r)))}{\sum_{k \in \varphi} \exp(\text{LeakyReLU}(\tilde{t}_r^T(W_r v_x^{r_k} + b_r)))} \quad (24)$$

Where $W_r \in \mathbb{R}^{2h+d}$ and $b_r \in \mathbb{R}$ are the learnable parameters. Weighted vectors of each type are summed to form an aggregated relation representation, computed as a weighted average of source-hidden features using a sigmoid function as in Eq. 25.

$$z_x^r = \sigma \left(\sum_{k \in \varphi} \alpha_x^{r_k} W_r C_x^{r_k} \right) \quad (25)$$

Finally, the node and its node representation are added to obtain the target representation as in Eq. (26),

$$NRP_s = e_x^r + e_x + f_x \quad (26)$$

Prediction layer (Fourth layer): The prediction task is framed as classification, categorizing future price trends into up, neutral, or down, with detailed task settings provided. The prediction network is a simple linear transformation layer defined in Eq. (27),

$$\hat{Y}_x = \text{Softmax} (w_j \times rep_s \times Y + b_j) \quad (27)$$

Where $w_j \in \mathbb{R}^{d \times 1}$ is a learnable parameter matrix and $b_j \in \mathbb{R}^d$ represents the bias matrix corresponding. Y is the number of market prediction where $Y = 4$ in this work. In this model, the cross-entropy loss is used to train full stock association in

the prediction layer. The loss function of this model is given in Eq. (28),

$$\mathcal{L}_{Node} = - \sum_{z \in C_i} \sum_{z=1} Y_{xz} \ln \hat{Y}_{xz} \quad (28)$$

Where, Y_{xz} represents the ground truth label of z^{th} movement class of stock x and C_i denotes the set of all data stocks. During training, the loss function L is minimized during training, enabling the model to handle numerous features, identify stock correlations, and select key indicators for effective prediction.

4. Result and discussion

4.1 Collected dataset

In this model, two datasets were used for the experiment evaluation, which are illustrated below. The financial and technical indices factor is listed in Table 4.

NSE [29]: The NSE is a leading stock exchange with a market capitalization exceeding US\$3.4 trillion ranking it among the world's top exchanges. It hosts over 2,000 companies, offers diverse financial instruments, with a 50-company index serving as a benchmark for the Indian market. This dataset provides detailed information on all NSE-listed companies, helping to economic growth. Some of the major NSE indices are Nifty 50, Nifty Next 50, Nifty Midcap 100, Nifty Smallcap 100, Nifty Bank.

BSE [30]: The BSE is a free-float market-weighted index of 30 major companies listed on the Bombay Stock Exchange, representing various sectors of the Indian economy. The dataset includes company logos, stock prices, and volume from Yahoo Finance, aiding investors in understanding stock behavior and

Table 4. Financial and Technical indices factors

No.	Financial Index Factor	Technical Index Factor
1	Price-to-Earnings (P/E) Ratio	Moving Averages (MA)
2	Earnings Per Share (EPS)	Relative Strength Index (RSI)
3	Price-to-Book (P/B) Ratio	Bollinger Bands
4	Dividend Yield	Moving Average Convergence Divergence (MACD)
5	Return on Equity (ROE)	Volume
6	Debt-to-Equity Ratio	Support and Resistance Levels
7	Market Capitalization	Fibonacci Retracement

forecasting trends, with BSE indices reflecting listed company performance. Some of the major BSE indices are BSE SENSEX (Sensitive Index), BSE 100, BSE 500, BSE MIDCAP and BSE SMALLCAP. **S&P 500 [31]:** It is the most famous financial benchmark in the world, tracking the performance of 500 large-cap companies listed on U.S. stock exchanges. As of December 31, 2020, more than \$5.4 trillion was invested in assets tied to the benchmark index. Due to the multiple share classes, (Alphabet's Class A (GOOGL) and Class C (GOOG), the index actually tracks 505 stocks. The index comprises most influential and widely held U.S. companies across key sectors, including Apple, Microsoft, Alphabet, Amazon and NVIDIA.

NASDAQ-100 [32]: It is a market cap-weighted index of 100 major non-financial NASDAQ-listed companies, with a strong focus on technology and innovation, serving as a key barometer for the U.S. tech sector. This dataset combines historical price data (open, high, low, close, volume) with key economic indicators to enable in-depth financial analysis and predictive modeling. The considered stock indexes of NASDAQ-100 are Adobe, Meta Platforms, Tesla, Netflix, PepsiCo.

4.2 Experimental setups

This section demonstrates the efficacy of the DDSPNet model its implementation in the Python 3.7. The experiments were conducted a computer with Intel® Core™ i5-4210 CPU@2.80GHz, 4GB RAM, 1TB hard disk under Windows 10 64-bit operating system. The performance evaluation is categorized into two main stages: (i) assessing the model's predictive performance (ii) evaluating the profitability of the investment strategies through metrics like return investment ratio and Sharpe Ratio (SR).

In this analysis, the proposed model is evaluated on standard models like LSTM, GRU, BERT, LSTM- GRU, GRU-BERT and BERT-LSTM. In addition, for the comparative evaluation, the DDSPNet model is compared with existing models like ML-GAT [16], MS-CNN [17], MS-SSA-LSTM [18], MEFT [19], HIMM [21] and SMP-DL [25] models. Table 5 depicts the hyper-parameters utilized for existing and proposed model. The formulas are briefly illustrated below.

Accuracy: It is calculated as the % of exactly predicted data to the total data tested.

$$\text{Accuracy} = \frac{\text{True Positive (TP)} + \text{True Negative (TN)}}{\text{TP} + \text{TN} + \text{False Positive (FP)} + \text{False Negative (FN)}} \quad (29)$$

Table 5. Hyperparameters of proposed model and existing models

Models	Parameters	Range
ML-GAT [16]	Time Series Length	50
	MLP - Hidden Layer	2
	LSTM Layer	128
	Optimizer	Adam
	Learning Rate	0.0005
	Activation Function	ReLU
	Epoch	100
	BERT - Batch Size	32
MS-CNN [17]	Learning Rate	0.001
	Optimizer	Adam
	Iterations	100
MS-SSA-LSTM [18]	Learning Rate	0.01
	Activation	Sigmoid
	Optimizer	Adam
MEFT [19]	Learning Rate	0.01
	Optimizer	Adam
HIMM [21]	Learning Rate	20^{-5}
	BERT - Hidden Layer	768
	Optimizer	AdamW
	Batch size	32
SMP-DL [25]	Epoch	50
	Optimizer	Adam
	Activation Function	ReLU
Proposed DDSPNet Model	Word embedding size	300
	No of LSTM Layers	2
	LSTM hidden Size	128
	GRU- hidden state	20
	Window Size	30
	Sequence Length	25
	Transformer Blocks	24
	BERT – Hidden Size	512
	No. of Attention Heads	21
	Graph convolutional layers	3
CNN, LSTM, GNN, GRU, GCN and DDSPNet	Learning rate	0.0001
	Epochs	125
	Batch size	64
	Activation Functions	ReLU
	Optimizer	Adam
	Loss function	Cross-entropy
	Momentum	0.97
	Dropout rate	0.5

In Eq. (29), TP is when the model correctly predicts a stock will rise; FN is when it predicts a drop but the stock rises. FP occurs when a rise is predicted but the stock falls, and TN is when a drop is correctly predicted.

F1-score: It is determined by

$$F1 - score = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (30)$$

Where,

$$Precision = \frac{TP}{TP+FP} \quad (31)$$

$$Recall = \frac{TP}{TP+FN} \quad (32)$$

Average Daily Return (ADR): It measures the percentage change in its value over a period, reflecting the gain or loss from investments.

$$RI_x^t = \frac{1}{|G^{t-1}|} \sum_{x \in |G^{t-1}|} \frac{Pf_x^t - Pf_x^{t-1}}{Pf_x^{t-1}} \quad (33)$$

In Eq. (33), G^{t-1} denotes a group of stocks included in portfolio at time $t-1$, Pf_x^t denotes the stock price x at time t , $|\cdot|$ represents the number of items in the set portfolio.

SR: It is an indicator that considers both return and risk and can be used to measure the performance of investment risk compared to return

$$SR = \frac{E[R_a] - R_F}{\sigma_P} \quad (34)$$

$$E[R_a] = \sum_{x=1}^n P_x \times R_x \quad (35)$$

Where, R_F represents Risk-free rate government bonds), $E[R_a]$ defines the expected return of the investment, P_x and R_x is the probability in stock cases x and n is the numbers of possible cases.

Mean Absolute Percentage error (MAPE): It is calculated as

$$MAPE = (\sum_{i=1}^N (|\hat{y}_i - y_i| / y_i)) / N \quad (36)$$

Root Mean Square Error (RMSE): It is calculated as

$$RSME = \left(\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \right)^{1/2} \quad (37)$$

Mean Absolute Error (MAE): It is calculated as

$$MAE = (\sum_{i=1}^N |\hat{y}_i - y_i|) / N \quad (38)$$

Coefficient of Determination (R^2): It is calculated as

$$R^2 = 1 - (\sum_{i=1}^N (y_i - \hat{y}_i)^2) / (\sum_{i=1}^N (y_i - \bar{y}_i)^2) \quad (39)$$

In Eqs. (36)-(39), N is the number of predicted samples, y_i and $\hat{y}_i$ represents the actual and predicted price respectively. MAPE, RMSE, and MAE calculate the distinction between the actual and predicted values, whose range of values is $[0, +\infty)$. The closer to 0, the superior the model's prediction ability. R^2 measures the degree of model fitting; its entire range is $[0, 1]$. The closer the R^2 is to 1, the better goodness-of-fit.

4.3 Performance analysis

For experimental purposes, this study divides the collected dataset into training and testing sets using a 70:30 ratios. Each fold includes a 29-month training set and a 7-month testing set, ensuring robust evaluation. In time series data, random splits risk data leakage, allowing the model to learn from future values. Although a single validation set is used, performance could improve with techniques like 4-fold cross-validation. A window size of 30 is employed to predict the next day's closing price movement using data from the preceding 30 days, capturing short-term trends while maintaining temporal integrity. Fig. 6 depicts the cross validation for period of 5 years (2020 -2024).

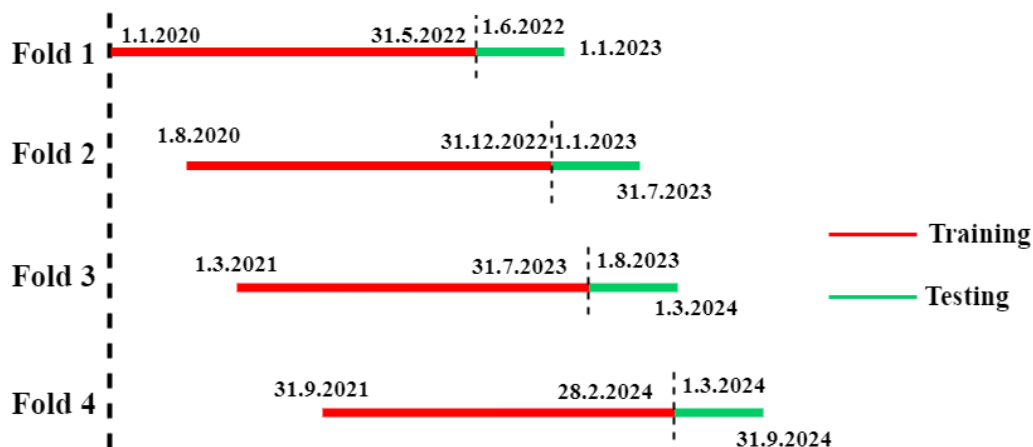


Figure. 6 4-fold cross validation for period of 5 years (2020 -2024)

4.4 Ablation study - performance analysis of individual and combined existing models with DDSPPnet

Table 6 presents the accuracy and f1-score comparison of various models on NSE and BSE datasets performing 4-fold cross validation. Similarly, Table 7 compares the ADR and SR performance of various models across five indices on NSE and BSE datasets. An ablation study using LSTM, GRU, BERT, LSTM-GRU, GRU-BERT, and the proposed DDSPPnet evaluates the individual impact of core architectural components on stock market prediction.

These models were evaluated on four major stock markets, including the NSE (Nifty 50), BSE (SENSEX), S&P 500 and NASDAQ-100, utilizing both prediction indicators (Accuracy and F1-score) and financial performance indicators (ADR and SR). Among the base models, LSTM achieved an average accuracy of 0.754 and a SR of 1.02, indicating strong

temporal modeling capabilities. GRU performed slightly better with 0.775 accuracy and a 1.18 Sharpe Ratio, likely due to its simpler architecture enabling more efficient learning of shorter sequences.

The LSTM-GRU hybrid model achieved an average accuracy of 0.834 and F1-score of 0.834, indicating synergy between two sequential models, but gains in financial measures were minor (SR ~1.26). GRU-BERT attained an average accuracy of 0.857, F1-score of 0.841, ADR of 0.38% and SR of 1.42. Moreover, the BERT-LSTM outperforms other models with average accuracy and F1-score of 0.873 and 0.870, respectively, with ADR and SR of 0.659 and 2.00. The DDSPPnet performed the best across all criteria, with an average accuracy of 0.904, an F1-score of 0.901, an ADR of about 0.44%, and a SR greater than 1.55. This shows the proposed model efficiently manages multiple variables and identifies key indicators for optimal stock movement prediction.

Table 6. Accuracy and F1-Score analysis of proposed and existing models for stock trend prediction

Model	Exchange	Accuracy-4-fold cross validation						F1-score-4-fold cross validation					
		Indexes					Avg.	Indexes					Avg.
		1	2	3	4	5		1	2	3	4	5	
LSTM	NSE	0.763	0.751	0.768	0.745	0.759	0.757	0.651	0.750	0.770	0.743	0.754	0.754
	BSE	0.772	0.760	0.766	0.749	0.754	0.760	0.710	0.765	0.771	0.743	0.746	0.747
	S&P 500	0.781	0.766	0.759	0.778	0.770	0.771	0.765	0.769	0.741	0.751	0.765	0.758
	NASDAQ-100	0.779	0.760	0.752	0.771	0.763	0.765	0.715	0.749	0.754	0.769	0.752	0.748
GRU	NSE	0.781	0.768	0.772	0.754	0.763	0.768	0.787	0.792	0.780	0.771	0.764	0.779
	BSE	0.788	0.775	0.760	0.777	0.769	0.774	0.768	0.803	0.798	0.780	0.774	0.785
	S&P 500	0.794	0.787	0.775	0.788	0.779	0.785	0.798	0.773	0.781	0.760	0.766	0.776
	NASDAQ-100	0.782	0.769	0.766	0.774	0.771	0.772	0.779	0.766	0.775	0.772	0.760	0.770
BERT	NSE	0.813	0.798	0.806	0.790	0.802	0.802	0.816	0.803	0.798	0.788	0.797	0.801
	BSE	0.821	0.809	0.812	0.795	0.807	0.809	0.805	0.826	0.816	0.809	0.799	0.811
	S&P 500	0.832	0.821	0.816	0.825	0.819	0.822	0.820	0.814	0.819	0.812	0.825	0.818
	NASDAQ-100	0.824	0.810	0.805	0.812	0.816	0.813	0.813	0.819	0.811	0.822	0.807	0.814
LSTM-GRU	NSE	0.837	0.821	0.828	0.810	0.824	0.824	0.830	0.810	0.825	0.812	0.823	0.820
	BSE	0.844	0.830	0.839	0.822	0.833	0.834	0.829	0.842	0.836	0.823	0.841	0.834
	S&P 500	0.855	0.842	0.836	0.848	0.844	0.845	0.847	0.849	0.841	0.835	0.845	0.843
	NASDAQ-100	0.848	0.832	0.827	0.837	0.838	0.836	0.838	0.849	0.831	0.846	0.842	0.841
GRU-BERT	NSE	0.861	0.849	0.844	0.835	0.848	0.847	0.853	0.840	0.839	0.830	0.841	0.841
	BSE	0.871	0.858	0.853	0.845	0.860	0.857	0.841	0.850	0.847	0.828	0.837	0.841
	S&P 500	0.883	0.871	0.862	0.875	0.867	0.872	0.865	0.851	0.843	0.840	0.844	0.849
	NASDAQ-100	0.874	0.862	0.853	0.861	0.864	0.863	0.863	0.848	0.851	0.856	0.850	0.854
BERT-LSTM	NSE	0.881	0.869	0.864	0.856	0.870	0.868	0.871	0.865	0.870	0.866	0.869	0.868
	BSE	0.889	0.878	0.871	0.863	0.881	0.876	0.889	0.878	0.875	0.861	0.863	0.873
	S&P 500	0.899	0.889	0.880	0.892	0.886	0.889	0.889	0.873	0.869	0.858	0.860	0.870
	NASDAQ-100	0.892	0.881	0.872	0.880	0.884	0.882	0.882	0.872	0.860	0.870	0.880	0.873
DDSPPnet	NSE	0.901	0.892	0.884	0.890	0.899	0.893	0.901	0.899	0.892	0.893	0.834	0.896
	BSE	0.913	0.902	0.894	0.900	0.911	0.904	0.911	0.900	0.896	0.891	0.908	0.901
	S&P 500	0.920	0.911	0.902	0.915	0.918	0.913	0.910	0.895	0.902	0.914	0.911	0.906
	NASDAQ-100	0.911	0.900	0.891	0.902	0.915	0.903	0.905	0.890	0.894	0.902	0.907	0.900

Table 7. ADR and SR analysis of proposed and existing models for stock trend prediction

Model	Exchange	ADR						SR					
		Indexes					Avg.	Indexes					Avg.
		1	2	3	4	5		1	2	3	4	5	
LSTM	NSE	0.023	-0.045	0.051	0.033	0.012	0.015	1.62	1.51	1.68	1.57	1.60	1.59
	BSE	-0.017	0.072	0.068	-0.013	0.043	0.030	1.64	1.68	1.71	1.63	1.70	1.66
	S&P 500	0.064	-0.038	0.012	0.057	-0.019	0.015	1.72	1.66	1.70	1.68	1.71	1.70
	NASDAQ-100	-0.029	0.084	-0.051	0.031	0.066	0.020	1.67	1.69	1.74	1.69	1.72	1.70
GRU	NSE	0.014	0.065	-0.059	0.024	-0.002	0.007	1.61	1.65	1.73	1.60	1.63	1.65
	BSE	-0.045	0.059	-0.032	0.067	0.004	0.010	1.67	1.71	1.76	1.64	1.71	1.69
	S&P 500	0.072	-0.009	0.031	0.008	0.065	0.033	1.72	1.73	1.78	1.70	1.74	1.73
	NASDAQ-100	0.0248	0.046	-0.066	0.033	-0.019	0.003	1.64	1.66	1.73	1.65	1.69	1.67
BERT	NSE	0.0352	0.079	-0.013	0.046	-0.005	0.028	1.71	1.76	1.78	1.73	1.75	1.75
	BSE	0.0643	-0.031	0.026	0.048	0.033	0.028	1.73	1.73	1.79	1.75	1.77	1.75
	S&P 500	-0.046	0.062	-0.008	0.056	-0.001	0.016	1.75	1.76	1.80	1.74	1.78	1.76
	NASDAQ-100	0.0528	-0.019	0.061	-0.034	0.074	0.027	1.76	1.78	1.82	1.78	1.79	1.78
LSTM-GRU	NSE	0.0125	0.047	0.028	0.055	-0.036	0.021	1.70	1.76	1.78	1.73	1.77	1.74
	BSE	0.0347	-0.042	0.065	0.016	0.044	0.023	1.71	1.75	1.79	1.74	1.76	1.75
	S&P 500	-0.0539	0.071	0.018	0.039	0.028	0.020	1.71	1.77	1.82	1.75	1.78	1.77
	NASDAQ-100	0.0617	0.015	-0.022	0.068	-0.016	0.021	1.77	1.85	1.83	1.79	1.81	1.79
GRU-BERT	NSE	0.0431	0.066	-0.011	0.051	0.036	0.037	1.74	1.78	1.83	1.76	1.79	1.73
	BSE	0.0725	-0.023	0.048	0.065	-0.012	0.030	1.76	1.80	1.84	1.77	1.79	1.79
	S&P 500	-0.011	0.055	0.069	-0.029	0.053	0.027	1.79	1.82	1.85	1.80	1.83	1.82
	NASDAQ-100	0.0586	-0.047	0.062	0.028	-0.022	0.015	1.72	1.79	1.81	1.77	1.82	1.78
BERT-LSTM	NSE	0.058	0.036	0.070	0.058	0.064	0.053	1.94	2.17	1.96	1.95	1.87	1.98
	BSE	0.075	0.013	0.061	-0.080	-0.071	0.057	2.02	1.99	1.97	2.06	2.05	2.02
	S&P 500	-0.087	0.062	-0.077	0.081	0.071	0.075	2.01	2.09	2.03	2.07	2.01	2.04
	NASDAQ-100	0.081	-0.073	0.082	-0.076	0.080	0.078	1.97	1.95	1.75	2.16	1.96	1.96
DDSPPnet	NSE	0.066	0.041	-0.089	0.031	-0.073	0.056	1.96	2.01	2.05	1.98	1.99	2.00
	BSE	0.084	-0.012	0.065	-0.089	0.072	0.059	1.99	1.97	1.97	2.08	2.02	2.03
	S&P 500	-0.09	0.073	0.086	0.063	-0.089	0.078	2.05	2.06	2.08	2.03	2.07	2.08
	NASDAQ-100	0.067	0.095	0.078	0.085	-0.095	-0.083	2.05	2.06	2.10	2.08	2.07	2.07

4.5 Comparison of actual vs. predicted opening and closing task across different exchanges

Table 8 demonstrates the effectiveness of the proposed DDSPPnet model in predicting stock market trends across three thresholds.

At $\pm 5\%$, the model predominantly predicted neutral ($\rightarrow$) trends across NSE, BSE, S&P 500 and NASDAQ-100, aligning well with actual movements due to limited volatility. At $\pm 2\%$, DDSPPnet showed improved sensitivity, accurately detecting downward trends ($\downarrow$) for volatile stocks like Adobe and Apple on specific dates. The most sensitive $\pm 0.5\%$ threshold enabled precise micro-trend predictions, capturing slight upward ($\uparrow$) and neutral/downward movements, especially for Apple and Adobe.

In Indian markets, where fluctuations rarely exceed $\pm 1\%$, the model consistently predicted neutral trends, reflecting robustness in stable environments.

DDSPPnet's fine-grained accuracy at $\pm 0.5\%$ benefits short-term traders, while its stable performance at $\pm 5\%$ suits long-term investors. Overall, the model adapts effectively across thresholds, accurately capturing trends and fluctuations in both INR and USD markets.

4.6 Comparison of inter-stock relation prediction

Table 9 depicts the comparison analysis of wikidata vs. MHVP+wikidata for inter-stock relation prediction using Accuracy and F1-score. In this analysis, the MHVP+wikidata outperforms wikidata across all exchanges. MHVP captures important temporal correlation between the stocks that static wikidata alone cannot process the task. Hence, combining MHVP+wikidata identifies relevant inter-stock dependencies for improving the stock prediction in multiple data.

Table 8. Ground truth stock price movements and prediction by proposed DDSPPnet model for different thresholds

Date	Stock Exchange	Previous Close	Opening	Closing	% Change (Actual Trend)	Predicted Values		
						±5% Threshold	±2% Threshold	±0.5% Threshold
12/04/2024	NSE (Nifty 50)	22753.80	22677.40	22519.40	-1.03% (↓)	→	→	↓
	BSE (SENSEX)	75038.15	74889.64	74244.90	-1.06% (↓)	→	→	↓
	S&P 500 (Apple)	175.04	174.26	176.55	+0.86% (↑)	→	→	↑
	NASDAQ-100 (Adobe)	484.28	477.95	474.09	-2.10% (↓)	→	↓	↓
16/05/2024	NSE (Nifty 50)	22200.55	22319.20	22403.85	+0.92% (↑)	→	→	↑
	BSE (SENSEX)	72987.03	73338.24	73663.72	+0.93% (↑)	→	→	↑
	S&P 500 (Apple)	189.72	190.47	189.84	+0.06% (→)	→	→	→
	NASDAQ-100 (Adobe)	485.35	484.70	482.88	-0.51% (↓)	→	→	↓
20/06/2024	NSE (Nifty 50)	23516.00	23586.15	23567.00	+0.22% (→)	→	→	→
	BSE (SENSEX)	77337.59	77554.83	77478.93	+0.18% (→)	→	→	→
	S&P 500 (Apple)	214.29	213.93	209.68	-2.15% (↓)	→	↓	↓
	NASDAQ-100 (Adobe)	527.66	0.00	522.95	-0.89% (↓)	→	→	↓
24/07/2024	NSE (Nifty 50)	24479.05	24444.95	24413.50	-0.27% (→)	→	→	→
	BSE (SENSEX)	80429.04	80343.38	80148.88	-0.35% (→)	→	→	→
	S&P 500 (Apple)	225.01	224.00	218.54	-2.88% (↓)	→	↓	↓
	NASDAQ-100 (Adobe)	532.15	543.80	531.04	-0.21% (→)	→	→	→
28/08/2024	NSE (Nifty 50)	25017.75	25030.80	25052.35	+0.14% (→)	→	→	→
	BSE (SENSEX)	81785.56	81779.84	81785.56	+0.00% (→)	→	→	→
	S&P 500 (Apple)	228.03	227.92	226.49	-0.68% (↓)	→	→	↓
	NASDAQ-100 (Adobe)	567.82	568.22	560.54	-1.28% (↓)	→	→	↓
03/09/2024	NSE (Nifty 50)	25278.70	25313.40	25279.85	+0.00% (→)	→	→	→
	BSE (SENSEX)	82559.84	82652.69	82555.44	-0.01% (→)	→	→	→
	S&P 500 (Apple)	220.85	228.55	222.77	+0.87% (↑)	→	→	↑
	NASDAQ-100 (Adobe)	574.41	576.30	571.04	-0.59% (↓)	→	→	↓

↑ (Up); ↓ (Down) → (Neutral); NSE and BSE currency- INR; S&P 500 and NASDAQ-100 currency- USD

Table 9. Analysis of wikidata vs. MHVP+wikidata for inter-stock relation prediction

Criteria	Metric	NSE (Nifty-50)	BSE (SENSEX)	S&P 500	NASDAQ-100
wikidata	Accuracy	91.12%	90.45%	91.80%	90.90%
MHVP+ wikidata		94.94%	93.86%	94.20%	93.03%
wikidata	F1-score	89.64%	88.55%	90.22%	88.98%
MHVP+ wikidata		93.89%	91.81%	93.10%	90.30%

Table 10. Computational efficiency of the different model on collected dataset

Dataset	Computational Metrics		LSTM-GRU	GRU-BERT	BERT-LSTM	DDSPPnet
NSE (Nifty-50)	Training time (sec)	CPU	3672	3347	3117	2892
		GPU	1003	953	939	859
	Inference time (sec)	CPU	379	361	328	274
		GPU	323	286	272	216
	Memory usage (MB)	CPU	5393	5139	5087	5074
		GPU	4374	4297	4197	4092
BSE (SENSEX)	Training time (sec)	CPU	3615	3469	3320	2785
		GPU	1017	970	917	835
	Inference time (sec)	CPU	376	347	332	290
		GPU	319	288	267	169
	Memory usage (MB)	CPU	5210	5070	4970	4895
		GPU	4407	4302	4191	3850
S&P 500	Training time (sec)	CPU	3672	3347	3120	2981
		GPU	1003	953	945	790
	Inference time (sec)	CPU	379	361	325	318
		GPU	323	286	267	203
	Memory usage (MB)	CPU	5393	5139	4784	5079
		GPU	4374	4297	4325	4197
NASDAQ-100	Training time (sec)	CPU	3672	3347	3120	2990
		GPU	1003	953	945	917
	Inference time (sec)	CPU	379	361	325	315
		GPU	323	286	267	198
	Memory usage (MB)	CPU	5393	5139	4980	5098
		GPU	4374	4297	4125	4197

4.7 Computational metrics

Table 10 compares the computational efficiency of the proposed DDSPPnet model with hybrid models such as LSTM-GRU, GRU-BERT and BERT-LSTM on the NSE (Nifty-50) and BSE (Sensex), S&P 500 and NASDAQ-100 datasets. The evaluation considers CPU and GPU performance in terms of training CPU and GPU performance in terms of training time, inference time per instance, and memory usage. Compared to other models, DDSPPnet demonstrates improved efficiency by reducing training time and inference time while maintaining balanced memory consumption. While, GCN model takes longer training and inference times compared to DDSPPnet on CPU and GPU. However, GGN consumes slightly lower memory usage than DDSPPnet model for larger datasets. The results suggest that DDSPPnet is more efficient in both speed and memory usage, making it a better choice for larger datasets and optimizes the computational performance.

4.8 Comparative analysis

Table 11 displays various stock prediction models exhibiting their distinct methods employing multiple datasets, methodology, and performance

criteria. The proposed DDSPPnet surpasses previous models in terms of accuracy, F1-score, and financial viability.

Unlike ML-GAT [16], which has an accuracy of less than 57%, DDSPPnet achieves accuracies of 0.893 (NSE), 0.904 (BSE), 0.913 (S&P 500) and 0.903 (NASDAQ-100). DDSPPnet includes numerous financial metrics (ADR, SR, MAPE, R^2 , RMSE, MAE, AR, and MCC) for a more thorough assessment, unlike MS-CNN [17], which just considers profits. Unlike MS-SSA-LSTM [18], which obtains high R^2 but concentrates on a single market, DDSPPnet generalizes across many stock datasets.

For example, DDSPPnet achieves F1-scores of 0.896 (NSE), 0.901 (BSE), 0.906 (S&P 500), and 0.900 (NASDAQ-100), while maintaining low MAPE values (e.g., 0.0137 for NSE, 0.1486 for BSE), and high R^2 values (up to 0.9883 for NSE and 0.9874 for S&P 500). MEFT [19] has an accuracy rate of 54.16% (AAPL) and 51.66% (AMZN), which is much lower than DDSPPnet's above 90% accuracy. HIMM [21], despite its high recall (95.19%), has a lesser accuracy (69.20%), while DDSPPnet balances all major metrics with MCC values of 0.1743 (NSE), 0.48 (BSE), 0.56 (S&P 500), and 0.473 (NASDAQ-100). Compared to SMP-DL [25], which obtains a high R^2 (0.9948), DDSPPnet achieves up to 0.9994 (NSE) and 0.9985 (BSE) in previous evaluations.

Table 11. Comparison of different stock prediction models

Models	Dataset	Train\Test Ratio on time period	Performance Metrics			
ML-GAT [16]	S&P 500 Index (423 stocks) and SI 300 Index (286 stocks). Historical price data, textual data (news) and cooperate relation data	-	Average Accuracy (S&P500) = 0.5085;	Average F1-Score (S&P500) = 0.5102;	ADR (S&P500) = 0.1193	Average SR (S&P500) = 1.8889;
			Average Accuracy (CSI 300) = 0.5698	Average F1-Score (CSI 300) = 0.5691	ADR (CSI 300) = 0.1334	Average SR (CSI 300) = 1.8994
MS-CNN [17]	Dow Jones (DJI), AAPLE (AAPL) and General Electric (GE) from the Yahoo Finance.	-	Annualized return (AR) (DJI) = 30.91	AR (AAPL) = 61.55	AR (GE) = 82.95	-
			SR (DJI) = 0.928	SR (AAPL) = 1.563	SR (GE) = 1.678	-
MS-SSA-LSTM [18]	China A-share market stocks with financial indexes (2016–2022) from Ruisi Financial trade and East Money forum posts for stocks' comment information	7:3	MAPE (Petrol china) = 0.018216;	RMSE (Petrol china) = 0.123077;	MAE (Petrol china) = 0.091298;	R² (Petrol china) = 0.956459;
			MAPE (Hifuture Technology) = 0.029589	RMSE (Hifuture Technology) = 0.157564	MAE (Hifuture Technology) = 0.114505	R² (Hifuture Technology) = 0.885653
MEFT [19]	KDD17 dataset	8:1:1	Accuracy (AAPL) = 54.16%	Accuracy (AMZN) = 51.66%	-	-
HIMM [21]	StockNet dataset	8:0:2	Accuracy = 69.20%;	Precision = 63.49 %	Recall = 95.19%	MCC = 0.43
SMP-DL [25]	AAPL dataset [36] and GOOG dataset	-	-	RMSE = 0.2883	MAE = 0.1856	R² = 0.9948
Proposed DDSPPnet	NSE	7:3	Average Accuracy = 0.893	Average F1-Score = 0.896	MCC = 0.1743	Average SR = 2.00
			-	ADR = 0.056	AR = 97.5	
			MAPE = 0.0137	RMSE = 0.095	MAE = 0.0698	R² = 0.9883
	BSE	7:3	Average Accuracy = 0.904	Average F1-Score = 0.901	MCC = 0.48	Average SR = 2.03
			-	ADR = 0.059	AR = 96.39	
			MAPE = 0.1486	RMSE = 0.0931	MAE = 0.0698	R² = 0.9874
	S&P 500	7:3	Average Accuracy = 0.913	Average F1-Score = 0.906	MCC = 0.56	Average SR = 2.08
			-	ADR = 0.078	AR = 95.59	
			MAPE = 0.1347	RMSE = 0.0957	MAE = 0.0643	R² = 0.9791
	NASDAQ-100	7:3	Average Accuracy = 0.903	Average F1-Score = 0.900	MCC = 0.473	Average SR = 0.07
			-	ADR = 0.083	AR = 96.89	
			MAPE = 0.1446	RMSE = 0.0965	MAE = 0.0615	R² = 0.9821

Furthermore, DDSPPnet demonstrates strong generalization across multiple stock indices, ensuring resilience in volatile markets and adaptability to diverse financial conditions. It offers greater profitability, with annualized returns (AR) up to 97.5 (NSE) and 96.89 (NASDAQ-100) while keeping low error rates as RMSE as low as 0.095 and MAE of just 0.0615 making it the most resilient and flexible model for stock prediction.

5. Conclusion

In this paper, DDSPPnet handles extensive features and determines effective stock correlations by integrating data from multiple sources. It extracts price data features using LSTM to capture long-term dependencies. Quantitative data features including financial and technical indices are extracted with GRU, while sentiment data features are obtained by BERT. Relational data is processed with an improved model using MVHP to identify stock correlations, which are combined with Wikidata's data. Finally, the integrated features are input into EMLGAM for the prediction of stock price. Finally, the test results prove that the proposed model achieves an average accuracy of 0.893, 0.904, 0.913, 0.903, along with an average F1-Score of 0.896, 0.901, 0.906, 0.900 on India's NSE and BSE, S&P 500) and NASDAQ-100 datasets respectively, outperforming LSTM, GRU, BERT, LST M-GRU, GRU-BERT and BERT-LSTM models.

Conflicts of Interest

The authors declare no conflict of interest.

Author Contributions

Conceptualization, methodology, software, validation, Gayathri; formal analysis, investigation, Devi Suganya; resources, data curation, writing—original draft preparation, Gayathri; writing—review and editing, Gayathri; visualization, supervision, Devi Suganya;

References

- [1] C. Challoumis, "The Role of National Governments, Domestic Economies, and Enforcement and Escape Savings in Economic Stability: Lessons from the Greek Economic Crisis", *Procedia Econ. Sci. Res.*, Vol. 11, pp. 213-229, 2024.
- [2] L. D. Oyeniyi, C. E. Ugochukwu, and N. Z. Mhlongo, "Analyzing the Impact of Algorithmic Trading on Stock Market Behavior: A Comprehensive Review", *World J. Adv. Eng. Technol. Sci.*, Vol. 11, No. 2, pp. 437-453, 2024.
- [3] P. Srinivasan and K. Prakasam, "Stock Market Development and Economic Growth in India: An Empirical Analysis", *Munich Pers. RePEc Arch.*, No. 1, 2014.
- [4] S. Rath, N. R. Das, and B. K. Pattanayak, "An Analytic Review on Stock Market Price Prediction Using Machine Learning and Deep Learning Techniques", *Recent Patents Eng.*, Vol. 18, No. 2, pp. 88-104, 2024.
- [5] O. Ican and T. B. Celik, "Stock Market Prediction Performance of Neural Networks: A Literature Review", *Int. J. Econ. Finance*, Vol. 9, No. 11, pp. 100-108, 2017.
- [6] W. Jiang, "Applications of Deep Learning in Stock Market Prediction: Recent Progress", *Expert Syst. Appl.*, Vol. 184, p. 115537, 2021.
- [7] M. Dadhich, M. S. Pahwa, V. Jain, and R. Doshi, "Predictive Models for Stock Market Index Using Stochastic Time Series ARIMA Modeling in Emerging Economy", *Adv. Mech. Eng.: Select Proc. CAMSE 2020*, pp. 281-290, Springer Singapore, 2021.
- [8] A. Jeribi and A. Ghorbel, "Forecasting Developed and BRICS Stock Markets with Cryptocurrencies and Gold: Generalized Orthogonal Generalized Autoregressive Conditional Heteroskedasticity and Generalized Autoregressive Score Analysis", *Int. J. Emerg. Mark.*, Vol. 17, No. 9, pp. 2290-2320, 2022.
- [9] M. Abdullah, Z. Sulong, and M. A. F. Chowdhury, "Explainable Deep Learning Model for Stock Price Forecasting Using Textual Analysis", *Expert Syst. Appl.*, Vol. 249, p. 123740, 2024.
- [10] Y. Zhu, "Stock Price Prediction Using the RNN Model", In: *Proc. of J. Phys. Conf. Ser.*, Vol. 1650, p. 032103, 2020.
- [11] J. Wang, S. Hong, Y. Dong, Z. Li, and J. Hu, "Predicting Stock Market Trends Using LSTM Networks: Overcoming RNN Limitations for Improved Financial Forecasting", *J. Comput. Sci. Softw. Appl.*, Vol. 4, No. 3, pp. 1-7, 2024.
- [12] X. Ji, J. Wang, and Z. Yan, "A Stock Price Prediction Method Based on Deep Learning Technology", *Int. J. Crowd Sci.*, Vol. 5, No. 1, pp. 55-72, 2021.
- [13] W. Long, Z. Lu, and L. Cui, "Deep Learning-Based Feature Engineering for Stock Price Movement Prediction", *Knowl.-Based Syst.*, Vol. 164, pp. 163-173, 2019.
- [14] I. K. Nti, A. F. Adekoya, and B. A. Weyori, "A Novel Multi-Source Information-Fusion Predictive Framework Based on Deep Neural

- Networks for Accuracy Enhancement in Stock Market Prediction”, *J. Big Data*, Vol. 8, No. 17, pp. 1-28, 2021.
- [15] V. Snášel, J. D. Velásquez, M. Pant, D. Georgiou, and L. Kong, “A Generalization of Multi-Source Fusion-Based Framework to Stock Selection”, *Inf. Fusion*, Vol. 102, p. 102018, 2024.
- [16] K. Huang, X. Li, F. Liu, X. Yang and W. Yu, “ML-GAT: A Multilevel Graph Attention Model for Stock Prediction”, *IEEE Access*, vol. 10, pp. 86408-86422, 2022, doi: 10.1109/ACCESS.2022.3199008.
- [17] K. Cui, R. Hao, Y. Huang, J. Li, and Y. Song, “A Novel Convolutional Neural Networks for Stock Trading Based on DDQN Algorithm”, *IEEE Access*, Vol. 11, pp. 32308-32318, 2023.
- [18] G. Mu, N. Gao, Y. Wang, and L. Dai, “A Stock Price Prediction Model Based on Investor Sentiment and Optimized Deep Learning”, *IEEE Access*, Vol. 11, pp. 51353-51367, 2023.
- [19] T. W. Lee, P. Teisseyre, and J. Lee, “Effective Exploitation of Macroeconomic Indicators for Stock Direction Classification Using the Multimodal Fusion Transformer”, *IEEE Access*, Vol. 11, pp. 10275-10287, 2023.
- [20] Y. Xu and Y. Zhang, “Enhancement Economic System Based-Graph Neural Network in Stock Classification”, *IEEE Access*, Vol. 11, pp. 17956-17967, 2023.
- [21] J. Choi, S. Yoo, X. Zhou, and Y. Kim, “Hybrid Information Mixing Module for Stock Movement Prediction”, *IEEE Access*, Vol. 11, pp. 28781-28790, 2023.
- [22] H. Xu, Y. Zhang, and Y. Xu, “Promoting Financial Market Development-Financial Stock Classification Using Graph Convolutional Neural Networks”, *IEEE Access*, Vol. 11, pp. 17956-17967, 2023.
- [23] K. V. Rao and B. V. Ramana Reddy, “HM-SMF: An Efficient Strategy Optimization Using a Hybrid Machine Learning Model for Stock Market Prediction”, *Int. J. Image Graph.*, Vol. 24, No. 2, p. 2450013, 2024.
- [24] R. Corizzo and J. Rosen, “Stock Market Prediction with Time Series Data and News Headlines: A Stacking Ensemble Approach”, *J. Intell. Inf. Syst.*, Vol. 62, No. 1, pp. 27-56, 2024.
- [25] W. M. Shaban, E. Ashraf, and A. E. Slama, “SMP-DL: A Novel Stock Market Prediction Approach Based on Deep Learning for Effective Trend Forecasting”, *Neural Comput. Appl.*, Vol. 36, No. 4, pp. 1849-1873, 2024.
- [26] Z. Yang, T. Zhao, S. Wang, and X. Li, “MDF-DMC: A Stock Prediction Model Combining Multi-View Stock Data Features with Dynamic Market Correlation Information”, *Expert Syst. Appl.*, Vol. 238, p. 122134, 2024.
- [27] Z. Zhang and R. Xing, “Multivariate Hawkes process allowing for common shocks”, *Stat. Probab. Lett.*, Vol. 216, p. 110270, 2025.
- [28] W. Li, R. Bao, K. Harimoto, D. Chen, J. Xu, and Q. Su, “Modeling the Stock Relation with Graph Network for Overnight Stock Movement Prediction”, In: *Proc. of 29th Int. Conf. Int. Joint Conf. Artif. Intell.*, pp. 4541-4547, 2021.
- [29] <https://www.kaggle.com/datasets/andrewmvd/india-stock-market>
- [30] <https://www.kaggle.com/datasets/ravisane1/5-year-bse-sensex-dataset/data>
- [31] https://www.kaggle.com/datasets/andrewmvd/sp-500-stocks?select=sp500_index.csv
- [32] <https://www.nasdaq.com/market-activity/index/ndx/historical>